

Fundamentos do RideReady

o manual visual

Do «nunca vi código» ao «sei explicar o projeto por dentro»: o que é programar, o que é a web, o que são C#, ASP.NET Core, Blazor, Razor, EF Core e o Identity — e como tudo isso se monta no RideReady, com os ficheiros e o código verdadeiros do projeto.

Há um fio condutor: a viagem de **um clique** («Exportar alunos para Excel»), seguida do browser à base de dados e de volta. Cada conceito é primeiro um **boneco**; o texto explica em detalhe logo a seguir.

Acompanha os documentos de defesa das duas features (Exportação Excel · Email transacional). Versão de estudo — junho de 2026.

.. Índice

PARTE I — COMEÇAR DO ZERO

- 01 O que é um programa
- 02 Variáveis e tipos
- 03 Métodos
- 04 Classes, objetos e DTOs
- 05 async/await e try/catch
- 06 Cliente e servidor
- 07 URLs e as 3 línguas do browser
- 08 Tabelas, chaves e SQL

PARTE II — AS FERRAMENTAS

- 09 .NET, C# e ASP.NET Core
- 10 O pipeline HTTP
- 11 Blazor: Server vs WASM
- 12 O circuito SignalR
- 13 Render modes (SSR vs Interactive)
- 14 Razor, símbolo a símbolo
- 15 EF Core: entidades e DbContext
- 16 EF Core: LINQ

17 EF Core: migrations

18 Identity: contas e hash

19 Identity: cookie, roles, claims

20 DI: tomada, ficha e quadro

21 Tempos de vida

22 Radzen e Serilog

PARTE III — O RIDEREADY POR DENTRO

23 Os três projetos

24 A árvore de pastas

25 Program.cs comentado

26 As duas bases de dados

27 A cadeia real, linha a linha

28 A viagem do clique (poster)

29 Segurança: as três cancelas

PARTE IV — PARA A DEFESA

30 Vinte perguntas e respostas

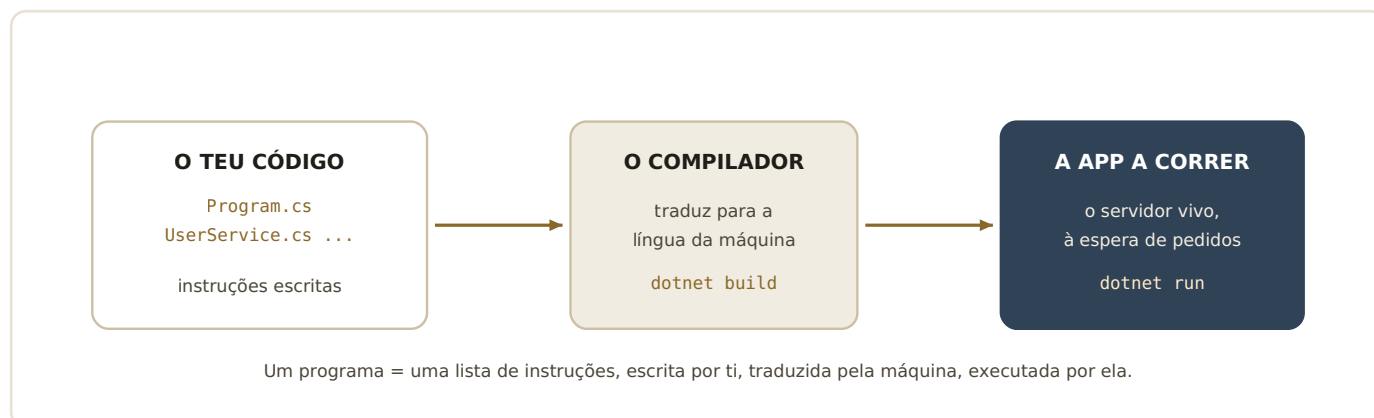
31 Glossário: 33 termos

32 A cábula final

PARTE I

Começar do zero — programar, a web e os dados

01 O que é um programa de computador



Do texto que escreves à aplicação viva: escrever, compilar, executar.

Um **programa** é, no fundo, uma lista de instruções muito precisas. Tu escreve-las em ficheiros de texto (o *código-fonte* — no teu caso, ficheiros `.cs` de C# e `.razor` de páginas). O computador não percebe esse texto diretamente: um **compilador** traduz tudo para a língua da máquina. É isso que acontece quando fazes `dotnet build` no Visual Studio (ou quando ele compila sozinho ao arrancar). Depois, `dotnet run --project RideReady` põe o resultado a **correr**: a partir daí existe um processo vivo no teu computador — o *servidor* — à escuta de pedidos do browser.

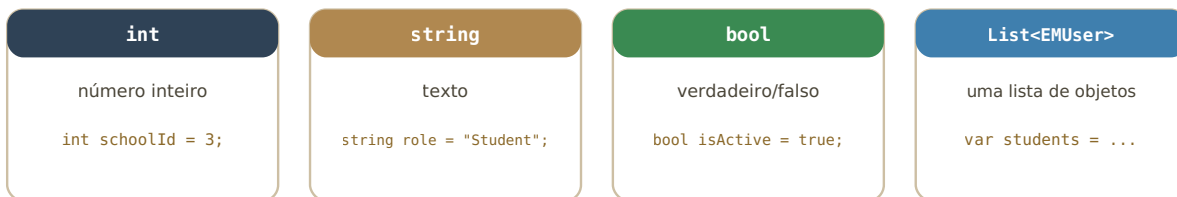
Três consequências práticas que já viveste no projeto: (1) se o código tem um erro de escrita, o compilador recusa — é o *build failed*, e a app nem arranca; (2) mudar um ficheiro nem sempre chega — às vezes é preciso **recompilar** (lembra-te dos `.razor.css` novos, que só entram com um restart completo); (3) o que corre é a *tradução*, não o teu texto — por isso é que «funciona no meu computador» depende do que foi compilado e com que configuração.

«O que acontece quando carregas no F5 do Visual Studio?»

Compila a solução (as três partes: RideReady, RepositoryLibrary, RideReadyAPI se incluída), e se tudo compilar, arranca o servidor e abre o browser apontado a ele. É o build + run num só gesto.

02 C# em miniatura: variáveis e tipos

Variáveis = caixas etiquetadas; o TIPO diz o que pode ir lá dentro



O C# é uma linguagem com tipos: a caixa 'int' recusa texto. Muitos erros são apanhados logo a compilar — antes de chegarem ao utilizador. 'var' não é um tipo: é pedir ao compilador para adivinhar o tipo pela direita do '='.

Quatro caixas que aparecem constantemente no teu código real.

Uma **variável** é uma caixa com etiqueta onde guardas um valor para usar mais à frente. Em C#, cada caixa tem um **tipo** — o formato do que lá pode entrar. No teu projeto vês a toda a hora: `int schoolId` (o número da escola), `string role` (o nome de um papel, como «Student»), `bool isActive` (se o utilizador está ativo), `byte[] fileBytes` (os bytes de um ficheiro — o teu Excel!), e coleções como `List<EMUser>` (uma lista de utilizadores).

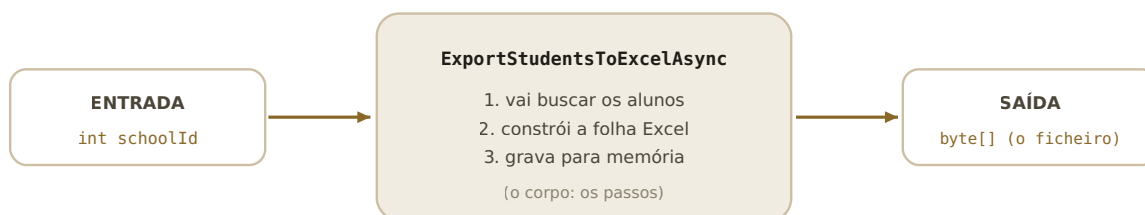
O tipo não é burocracia: é uma *rede de segurança*. Se tentares meter texto numa caixa de inteiros, o compilador recusa logo — apanhas o erro segundos depois de o escrever, e não dias depois com a app a rebentar à frente de alguém. Quando vires `var`, é só açúcar: o compilador deduz o tipo sozinho a partir do lado direito (`var students = await ...` é uma `List<EMUser>` porque o método devolve isso).

«O que é o EMUser? E o byte[]?»

EMUser é um tipo criado no vosso projeto (a classe que representa um utilizador — ver secção 04). `byte[]` lê-se «array de bytes»: uma sequência de uns-e-zeros em bruto, perfeita para representar um ficheiro em memória, como o Excel antes do download.

03 C# em miniatura: métodos

Um método = uma máquina com entrada e saída



Chamar o método = pôr a máquina a trabalhar:
`var bytes = await _export.ExportStudentsToExcelAsync(schoolId);`

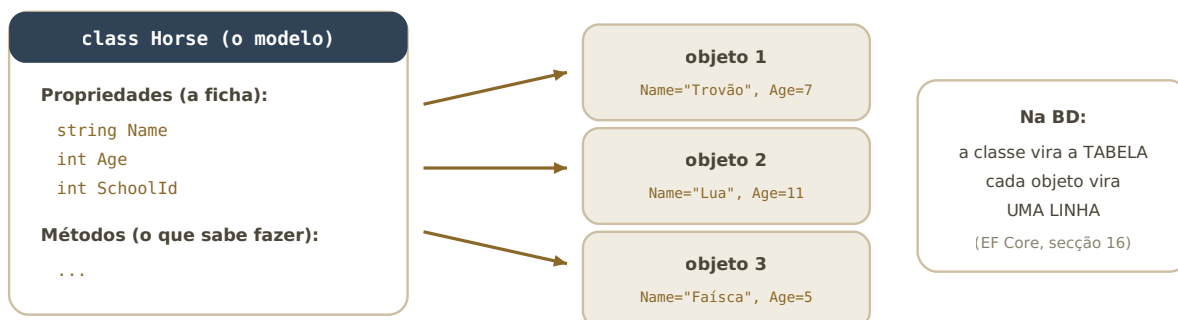
O teu método de exportação visto como máquina: entra um `schoolId`, sai um ficheiro.

Um **método** é um bloco de instruções com nome, que podes mandar executar quando quiseres («chamar o método»). Tem três partes: os **parâmetros** (o que entra), o **corpo** (os passos) e o **tipo de retorno** (o que sai). A assinatura real do teu serviço de Excel diz tudo: `Task<byte[]> ExportStudentsToExcelAsync(int schoolId)` — entra o número de uma escola, sai (uma promessa de) um array de bytes, o ficheiro.

Os métodos existem para *dar nome a um trabalho* e poder repeti-lo sem copiar código. Repara como os nomes no projeto são frases: `GetUsersBySchoolAndRole`, `SendConfirmationLinkAsync` — lê-se o que fazem. O sufixo `Async` e o `Task<...>` pertencem ao mundo do «assíncrono», que vem já a seguir (secção 05).

04 C# em miniatura: classes, objetos e DTOs

Classe = a ficha-modelo · Objetos = os cavalos reais nas boxes



No projeto: EMUser, Horse, Lesson, School, Purchase... são classes — as 'entidades'. Cada utilizador/cavalo/aula concreto é um objeto.

Uma classe e três objetos criados a partir dela — e a ponte para a base de dados.

Uma **classe** é um modelo: descreve que dados uma «coisa» tem (as **propriedades**) e o que sabe fazer (os métodos). Um **objeto** é um exemplar concreto criado a partir desse modelo. A classe `Horse` diz que todo o cavalo tem nome, idade, escola; o «Trovão, 7 anos, escola 3» é um objeto. No teu projeto, estas classes-modelo chamam-se **entidades** e vivem na RepositoryLibrary: `EMUser`, `Horse`, `Lesson`, `School`, `Purchase`...

Há ainda um parente que vais ver muito: os **DTOs** (*Data Transfer Objects*), como o `AdminUserListItemDto` da tua página de utilizadores. São classes «de transporte»: levam só os campos de que um ecrã precisa (nome, email, papel, estado), em vez de arrastar a entidade inteira com dados sensíveis (NIF, morada...) para onde não é preciso. É uma prática de segurança e de arrumação.

«Qual a diferença entre classe e objeto?»

A classe é a ficha-modelo (escreve-se uma vez, no código); os objetos são os exemplares vivos criados a partir dela durante a execução — zero, um ou mil. A classe `Horse` é uma; os cavalos da escola são muitos.

05 C# em miniatura: async/await e try/catch

async/await: não ficar espedado à espera

SEM await (bloqueante)

o empregado fica parado ao pé da mesa
até a cozinha acabar — ninguém mais é atendido

COM await (assíncrono)

o pedido segue; o empregado vai atender outros;
quando o prato está pronto, retoma onde ficou

try/catch: a rede de segurança

```
try { await client.SendMailAsync(message); }  
catch (Exception ex) { _logger.LogError(ex, "Falha..."); throw; }
```

tenta; se rebentar, apanha o erro, regista-o no log, e decide o que fazer com ele

As duas ferramentas de robustez que aparecem em todo o teu código.

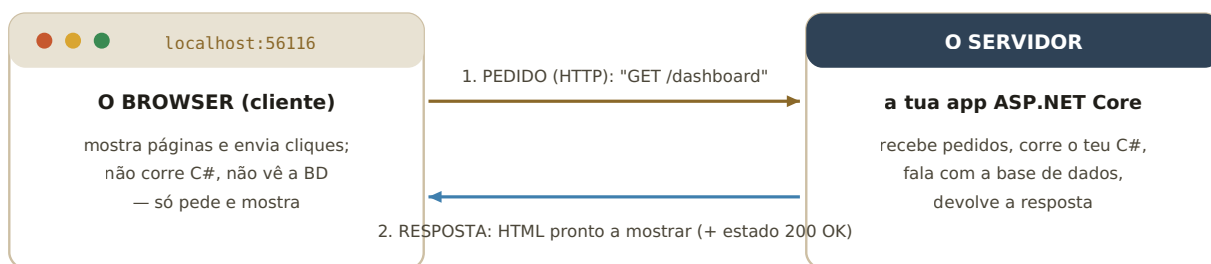
Falar com a base de dados, enviar um email, gravar um ficheiro — tudo isso *demora* (milissegundos que, para um servidor, são uma eternidade). O C# resolve com **async/await**: um método `async` devolve uma `Task` («uma promessa de resultado»), e o `await` diz «quando isto acabar, continua daqui — mas entretanto liberta o trabalhador para servir outros pedidos». É por isso que praticamente todos os métodos do projeto têm o sufixo `Async` e são chamados com `await`. Sem isto, cada utilizador à espera da BD bloquearia um trabalhador do servidor inteiro.

Quando algo corre mesmo mal (a BD caiu, o servidor de email não responde), o C# lança uma **exceção** — um grito de erro que sobe pela cadeia de chamadas até alguém o apanhar. O **try/catch** é o apanhador: no `try` vai o código arriscado; no `catch` decides o que fazer — registar no log (`_logger.LogError`), avisar o utilizador (como fazes no export: `NotificationService.Notify(...Erro...)`), ou relançar com `throw` para quem está acima decidir. Vais ver os três padrões nos teus dois documentos de feature.

Já viveste isto: o bug do `.Result` que corrigiste no export era exatamente isto — `.Result` é a forma bloqueante (o empregado espedado), e trocaste-a por `await`, a forma certa.

06 Como funciona a web: cliente e servidor

A dança de sempre: pedido e resposta



Enquanto desenvolves, cliente e servidor estão na MESMA máquina (localhost). Em produção, o servidor está noutro sítio — a dança é igual.

O modelo cliente-servidor: tudo na web é uma troca de pedidos e respostas.

A web inteira assenta neste vaivém: um **cliente** (o browser) faz um **pedido**, um **servidor** devolve uma **resposta**. O idioma desta conversa chama-se **HTTP**. Um pedido tem um *verbo* (GET = «dá-me», POST = «recebe isto») e um *caminho* (`/dashboard`, `/admin/users`); a resposta traz um *código de estado* (200 = OK, 404 = não existe, 500 = rebentou no servidor) e o conteúdo.

Quando escreves `localhost:56116` estás a dizer «o servidor é esta minha própria máquina, na porta 56116». A porta é como a extensão de telefone dentro do mesmo edifício — a tua app está à escuta nela. Repara que isto explica os teus testes: o `Ctrl+F5` força o browser a pedir tudo de novo em vez de usar a cópia local (a *cache*) — foi a cache que te pregou as partidas do favicon.

«Onde entra o HTTPS?»

É o HTTP com encriptação: a conversa vai cifrada para ninguém a ler pelo caminho. No Program.cs tens `app.UseHttpsRedirection()` — qualquer pedido em HTTP é empurrado para HTTPS.

07 URLs e as três línguas do browser



Um URL desmontado, e as três línguas que chegam ao browser.

O browser só percebe três línguas. **HTML** descreve o conteúdo («isto é um título, isto é um botão»). **CSS** descreve o aspeto («os títulos são navy, a 4.2rem») — é o que tens estado a escrever no `rideready-theme.css` e nos `.razor.css`. **JavaScript** é código que corre *dentro* do browser, para ações locais — no teu projeto há pouquíssimo, de propósito: o Blazor deixa-te fazer quase tudo em C#. A exceção honrosa é a função `downloadFileFromBytes` no `App.razor`, porque *disparar um download* é um gesto que só o browser pode fazer — e aí o C# pede ao JS (chama-se *JS Interop*, e vais defendê-la no documento do Excel).

E o `@page "/admin/users"` nas tuas páginas? É a ligação entre o *caminho* do URL e o ficheiro `.razor` que responde por ele — o «encaminhamento» (*routing*). Quando o browser pede `/admin/users`, o Blazor sabe que é a `AdminUsers.razor` que atende.

08 Bases de dados: tabelas, chaves e SQL

Uma base de dados relacional = tabelas que se apontam umas às outras



chave estrangeira: o SchoolId do cavalo aponta para o Id da escola

Falar com a BD: SQL

```
SELECT * FROM Horses WHERE SchoolId = 3;
```

O Id de cada linha é a CHAVE PRIMÁRIA: o número único que a identifica. As setas entre tabelas (chaves estrangeiras) são as 'relações' do nome relacional.

Tabelas, linhas, chaves — e a pergunta em SQL que devolve os cavalos da escola 3.

Uma **base de dados relacional** (como o teu SQL Server) guarda dados em **tabelas**: colunas com nome e tipo, linhas com os registos. Cada linha tem uma **chave primária** (o **Id**) que a identifica sem ambiguidade. E as tabelas relacionam-se por **chaves estrangeiras**: o cavalo «Trovão» guarda **SchoolId = 3**, que aponta para a linha 3 da tabela **Schools** — assim não se repete o nome da escola em cada cavalo; aponta-se.

A língua para perguntar e mandar é o **SQL**: **SELECT** (dá-me), **INSERT** (acrescenta), **UPDATE** (altera), **DELETE** (apaga). A boa notícia para ti: *quase nunca escreves SQL* no projeto — o EF Core (secção 16) traduz o teu C# em SQL por ti. Mas saber que isto existe por baixo é essencial para perceberes o que ele anda a fazer — e para responderes quando o professor perguntar «que SQL é gerado por esta consulta?».

«Porque se chama 'relacional'?»

Pelas relações entre tabelas (as chaves estrangeiras). É o que permite dizer 'os cavalos DA escola 3' sem duplicar dados — junta-se a informação pelas chaves.

PARTE II

As ferramentas do projeto — ASP.NET Core, Blazor, EF Core, Identity, DI

09 .NET, C# e ASP.NET Core: quem é quem

As camadas da plataforma — cada uma assenta na de baixo

a tua app RideReady vive aqui em cima



Blazor · EF Core · Identity — as especialidades

ASP.NET Core — a framework web (a estrutura da casa)

C# — a linguagem que escreves

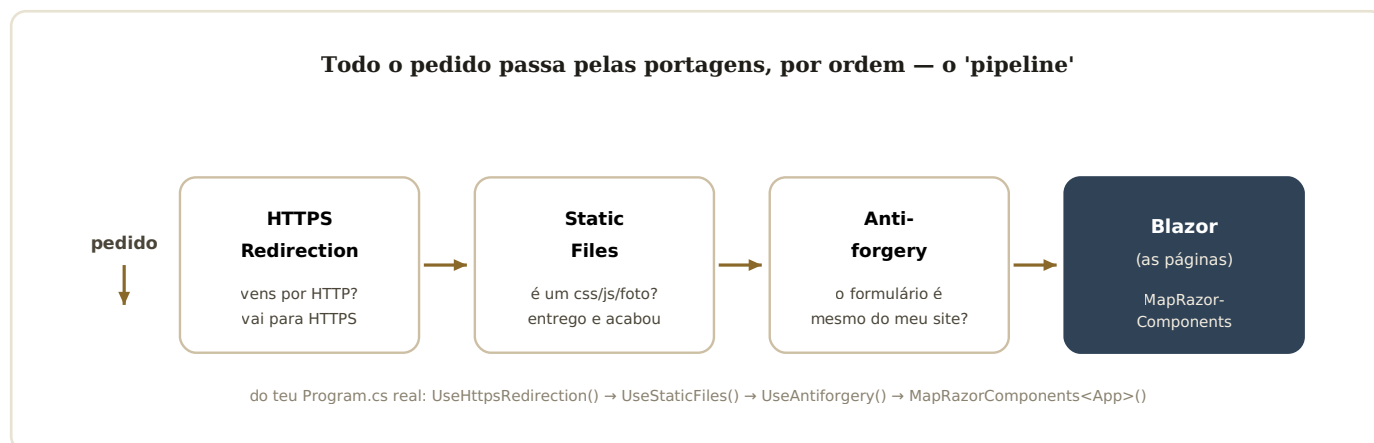
.NET 8 — a plataforma base (o terreno)

O empilhado: .NET por baixo de tudo; a tua app no topo.

.NET é a plataforma da Microsoft: o conjunto de máquinas e bibliotecas onde o C# corre (o vosso projeto usa a versão 8). **ASP.NET Core** é a parte dessa plataforma especializada em *web*: trata de receber pedidos HTTP, segurança, configuração, logging — a canalização que nenhuma app quer escrever do zero. E por cima vêm as especialidades que usas: **Blazor** (interfaces), **EF Core** (base de dados), **Identity** (contas).

O ponto de encontro de tudo é o `Program.cs`: o ficheiro que corre *primeiro* quando a app arranca, e onde se ligam as peças. Vamos lê-lo bloco a bloco na secção 22 — mas guarda já a ideia: *cada `builder.Services.Add...` é ligar uma peça à corrente.*

10 O pipeline HTTP: as portagens do servidor



O caminho de um pedido dentro do servidor, com as portagens do teu Program.cs.

Antes de um pedido chegar às tuas páginas, atravessa uma fila de **middleware** — pequenas portagens, cada uma com uma função, *pela ordem em que estão no Program.cs*. No teu projeto: `UseHttpsRedirection` (força a versão cifrada), `UseStaticFiles` (se o pedido é um ficheiro estático — o teu `rideready-theme.css`, as fotos dos cavalos — entrega-o já, sem acordar o resto da app), `UseAntiforgery` (proteção contra formulários forjados por outros sites) e finalmente `MapRazorComponents<App>()` — a entrada nas páginas Blazor.

A *ordem importa* e é pergunta clássica: os ficheiros estáticos vêm antes do resto porque são o caso mais frequente e mais barato; a segurança vem antes das páginas porque tem de inspecionar tudo o que lá chega.

11 Blazor: o que é, e Server vs WebAssembly

Os dois sabores do Blazor — o vosso é o da direita

Blazor WebAssembly

o C# é DESCARREGADO e corre
DENTRO do browser

- + funciona offline
- download inicial pesado
- SEM acesso direto à BD
(teria de chamar uma API)

Blazor Server ← o RideReady

o C# corre no SERVIDOR;
o browser liga-se por um fio (SignalR)

- + acesso direto a serviços e BD
- + arranque leve; código protegido
- precisa de ligação permanente
(se o fio cai, a página 'congela')

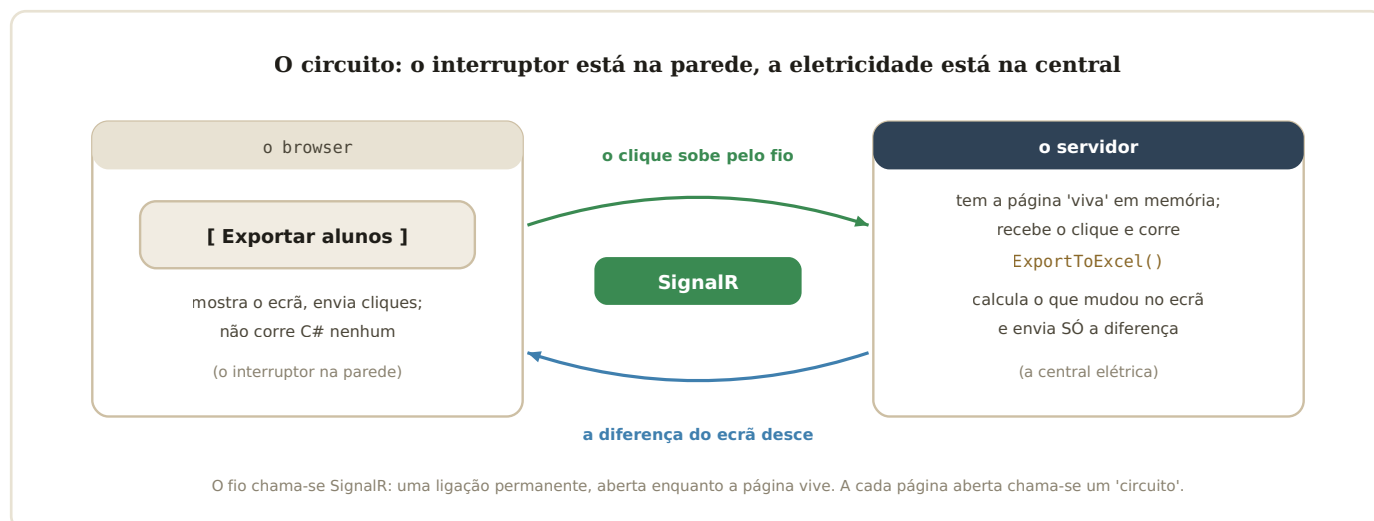
A escolha de Server é defensável: app de gestão interna, sempre ligada, com muito acesso a dados — exatamente o ponto forte do Server.

WebAssembly corre no browser; Server corre no servidor. O RideReady é Server.

Blazor é a tecnologia do ASP.NET Core para construir interfaces web inteiras em **C#** — páginas, botões, formulários — sem escrever JavaScript para a lógica. As páginas são os ficheiros **.razor**. Existem dois modos de execução, e a escolha define a arquitetura toda: no **WebAssembly**, o C# compilado é descarregado e corre dentro do browser; no **Server** — o vosso — o C# corre no servidor e o browser mantém uma ligação viva com ele.

Para o RideReady, Server é a escolha natural e deves dizê-lo com confiança: é uma aplicação de gestão (usada com ligação à internet), que toca constantemente na base de dados — e no modo Server as páginas estão «ao lado» dos serviços e da BD, podendo usá-los diretamente por injeção (`@inject`). No WebAssembly, cada acesso a dados teria de atravessar uma API pública — mais peças, mais superfície de ataque.

12 O circuito: SignalR por dentro



Cada clique viaja pelo fio; o C# corre do outro lado; volta só a diferença do ecrã.

No Blazor Server, quando abres uma página interativa, o servidor cria um **circuito**: mantém em memória uma cópia «viva» da página, ligada ao teu browser por **SignalR** (uma ligação em tempo real, sempre aberta). Quando clicas, o clique sobe pelo fio; o servidor corre o método C# correspondente; compara o ecrã antes/depois; e devolve só a *diferença*, que o browser aplica. É por isso que parece instantâneo sem nunca recarregar a página.

Este desenho explica vários fenómenos que já viveste: o aviso de «reconectar» quando o servidor reinicia (o fio caiu, o circuito morreu); o porquê de `@inject` funcionar nas páginas (elas vivem no servidor, ao pé dos serviços); e o porquê de cada utilizador ter os seus próprios serviços *Scoped* (um por circuito — secção 19).

«O que acontece se 50 alunos abrirem a app ao mesmo tempo?»

O servidor mantém 50 circuitos — 50 cópias vivas de páginas em memória, cada uma com a sua ligação SignalR e os seus serviços *Scoped*. É o custo do Server: memória no servidor proporcional aos utilizadores ligados.

13 Render modes: SSR vs InteractiveServer

Dois modos de 'desenhar' uma página — e o RideReady usa os dois

SSR (estático)

o servidor desenha o HTML,
entrega, e ESQUECE

sem circuito, sem estado

**usado em: Login, Register,
ForgotPassword, a tua Landing**

(formulários clássicos: submeter e seguir)

InteractiveServer

o servidor desenha E mantém
a página viva (circuito SignalR)

botões/cliques sem recarregar

**usado em: AdminUsers, dashboards,
lessons — tudo o que é 'app'**

(declara-se com `@rendermode InteractiveServer`)

Regra do projeto que aprendeste à força: as páginas do Identity têm de ficar SSR — torná-las interativas parte os serviços de conta (`NullReferenceException`).

SSR entrega e esquece; InteractiveServer mantém o circuito vivo.

Nem todas as páginas precisam do circuito. O Blazor moderno deixa escolher por página o **render mode**: **SSR** (*static server-side rendering*) — o servidor gera o HTML, entrega, fim; ou **InteractiveServer** — além de gerar, mantém a página viva com SignalR para reagir a cliques. A regra prática do vosso projeto: páginas «de formulário e segue» (login, registo, recuperar password — e a tua landing) são SSR; páginas «de aplicação» (grelhas, dashboards, botões que fazem coisas) são InteractiveServer.

E há uma razão dura que tu própria descobriste a debugar: as páginas do **Identity** dependem de serviços desenhados para o ciclo pedido-resposta clássico; postas em modo interativo, rebentam (`NullReferenceException`). É uma excelente história para a defesa — mostra que percebes a diferença entre os modos *na prática*, não só na teoria. Nota o detalhe técnico: o JS Interop (como o download do Excel) só funciona em páginas interativas — é por isso que a `AdminUsers.razor` declara `@rendermode InteractiveServer`.

14 Razor: a sintaxe, símbolo a símbolo

Um ficheiro .razor desmontado — a tua AdminUsers.razor

```
@page "/admin/users"
@rendermode InteractiveServer
@inject IStudentExportService _export
@inject IJSRuntime JSRuntime

<h1>User Management</h1>
@if (_loading) { <p>Loading...</p> }
<RadzenButton Text="Exportar..."
Click="@({args => ExportToExcel()})" />

@code {
    private bool _loading;
    private async Task ExportToExcel()
    { ... }
}
```

a morada e o modo

@page liga o URL a esta página

@inject — «tragam-me...»

pede ferramentas registadas na DI

a parte visível

HTML + componentes; @if/@foreach misturam C# no meio

@code — o cérebro

campos e métodos C# da página; os handlers dos cliques vivem aqui

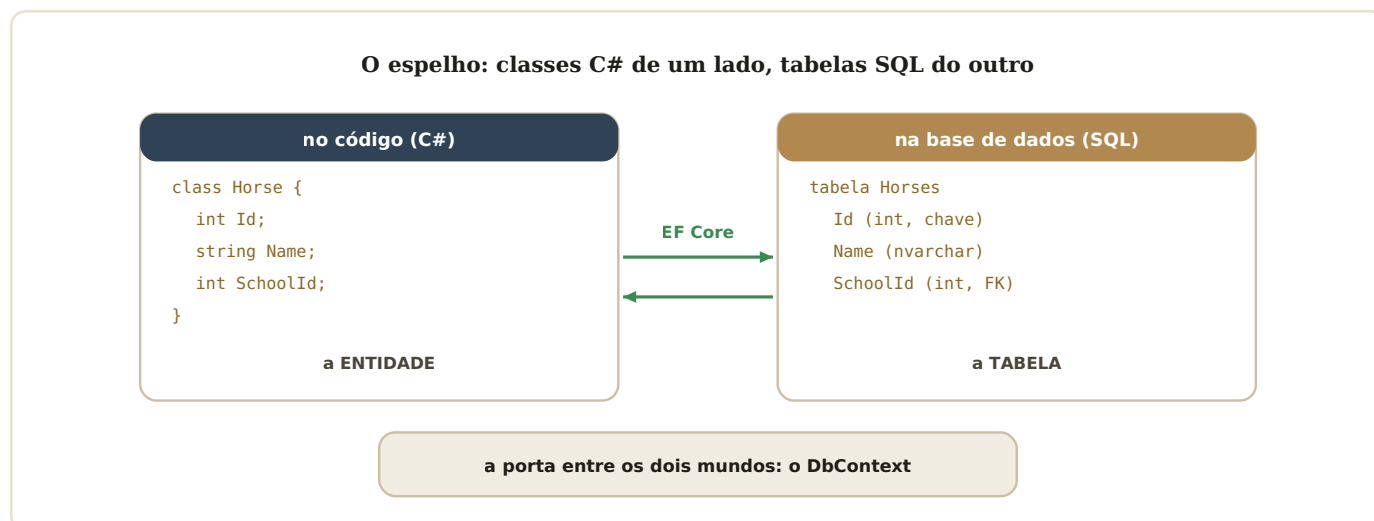
Os quatro andares de qualquer página: morada, injeções, HTML, código.

Razor é a sintaxe que mistura HTML e C# no mesmo ficheiro: tudo o que começa por `@` é C# embebido. O vocabulário essencial, com exemplos do projeto: `@page` (a morada no site), `@inject` (pede uma ferramenta à DI), `@code { }` (o C# da página), `@if / @foreach` (decidir e repetir no meio do HTML — é assim que a grelha mostra uma linha por utilizador), `@bind` (liga um campo de formulário a uma variável, nos dois sentidos), e os eventos — `Click="@({args => ExportToExcel()})"` — que ligam um gesto do utilizador a um método do `@code`.

Há ainda os **componentes**: páginas e pedaços de página são todos componentes Razor, e usam-se como etiquetas HTML — `<RadzenButton />`, `<UserForm />`, `<AuthorizeView>`. Componentes podem receber parâmetros (como o `User="editingUser"` do teu formulário) e conter outros componentes. A tua app é uma árvore de componentes, com o `App.razor` na raiz e o `MainLayout / EmptyLayout` como molduras.

Detalhe que já dominas: os ficheiros `Home.razor.css` e afins são CSS 'scoped' — válido só para o componente com o mesmo nome. E aprendeste a exceção à força: o scope não entra em componentes filhos (foi o caso `NavLink` vs na tua landing).

15 EF Core (1/3): entidades e DbContext



Cada entidade espelha uma tabela; o DbContext é a porta que as liga.

O **Entity Framework Core** (EF Core) é o tradutor entre o mundo dos objetos C# e o mundo das tabelas SQL — um *ORM* (Object-Relational Mapper). As tuas **entidades** (Horse, Lesson, EMUser...) espelham tabelas; cada objeto corresponde a uma linha. E a **porta** entre os dois mundos é o **DbContext**: uma classe que representa *uma* base de dados e expõe as tabelas como coleções (`context.Horses`, `context.SchoolUsers`) que consultas em C#.

O vosso projeto tem **dois** DbContexts, porque tem duas bases de dados (secção 23): o `AppIdentityDbContext` (contas) e o `RideReadyDbContext` (a escola). Repara no topo do teu `UserRepository`: ele recebe *ambos* no construtor — porque há perguntas que precisam dos dois mundos («que utilizadores [Identity] pertencem a esta escola [RideReady]?»).

16 EF Core (2/3): LINQ, a pergunta em C#

Uma consulta LINQ é uma linha de triagem



Importante: até ao ToListAsync NADA foi à base de dados.

O LINQ vai construindo a pergunta; só o ToListAsync/FirstOrDefaultAsync a envia — e é nesse momento que o EF Core gera e executa o SQL.

Where filtra, Select dá forma, ToListAsync dispara — só aí nasce o SQL.

LINQ é a forma de fazer perguntas a coleções em C#, e o EF Core traduz essas perguntas em SQL. Os três verbos que mais vêes no projeto: `.Where(...)` (filtra — «só os desta escola»), `.Select(...)` (transforma — «só quero o UserId de cada um»), e os executores `.ToListAsync()` / `.FirstOrDefaultAsync()` (corre a consulta e traz os resultados). As condições escrevem-se com *lambdas* — `su => su.SchoolId == schoolId` lê-se «para cada su, verifica se o SchoolId dele é o pedido».

O detalhe que rende pontos: a consulta é *preguiçosa*. Encadear `Where` e `Select` só vai *compondo* a pergunta; nada toca na BD até um executor. É por isso que o EF consegue gerar *um único* SQL eficiente com os filtros todos — em vez de trazer a tabela inteira e filtrar em memória.

«Que SQL gera o teu Where?»

Para `.Where(su => su.SchoolId == 3).ToListAsync()`: `SELECT * FROM SchoolUsers WHERE SchoolId = 3` — o filtro corre na base de dados, que é onde deve correr.

17 EF Core (3/3): migrations, o diário de obras

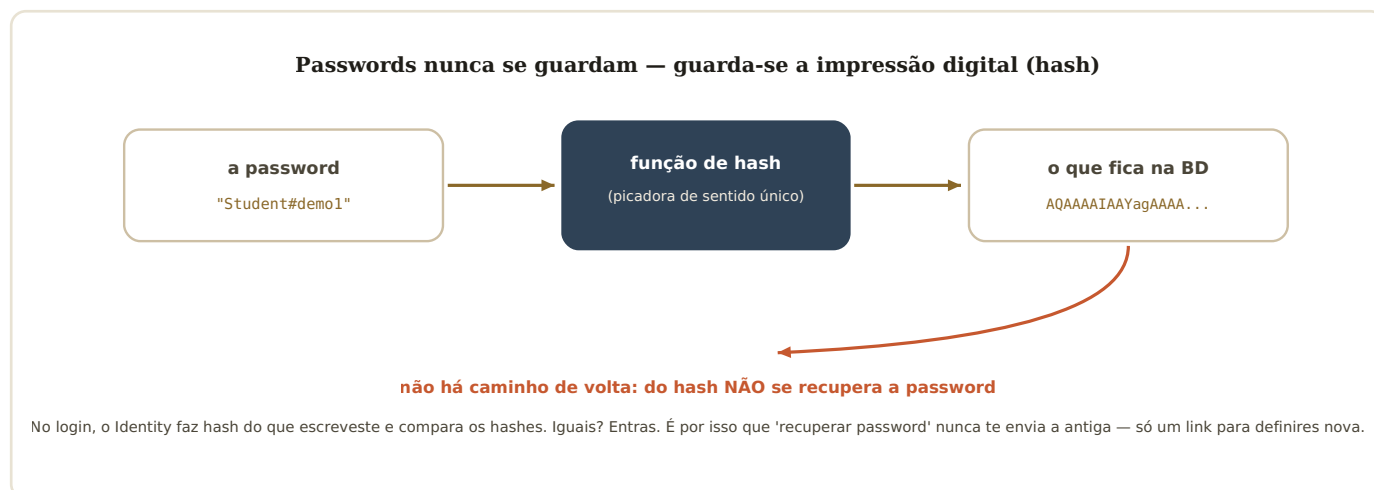


Cada mudança às entidades gera uma migration; o update aplica as que faltam.

Quando uma entidade muda (nova propriedade, nova tabela), a BD tem de acompanhar. As **migrations** são esse diário: cada mudança gera um ficheiro com as instruções de obra («adiciona a coluna NIF»), datado e ordenado. O comando `dotnet ef database update` compara o diário com a BD real e aplica o que falta. A BD guarda o registo do que já foi aplicado numa tabela própria — e aqui está um detalhe vosso para a defesa: como têm *duas* BDs, têm *duas* histórias separadas, `__EFMigrationsHistory_Identity` e `__EFMigrationsHistory_RideReady` (configurado no Program.cs, vais vê-lo na secção 22).

É também por isto que o guia de arranque para a tua explicadora manda correr *dois* updates, um por contexto — sem eles, a app arranca mas cai à primeira consulta, porque as tabelas não existem.

18 Identity (1/2): contas, registo e o hash

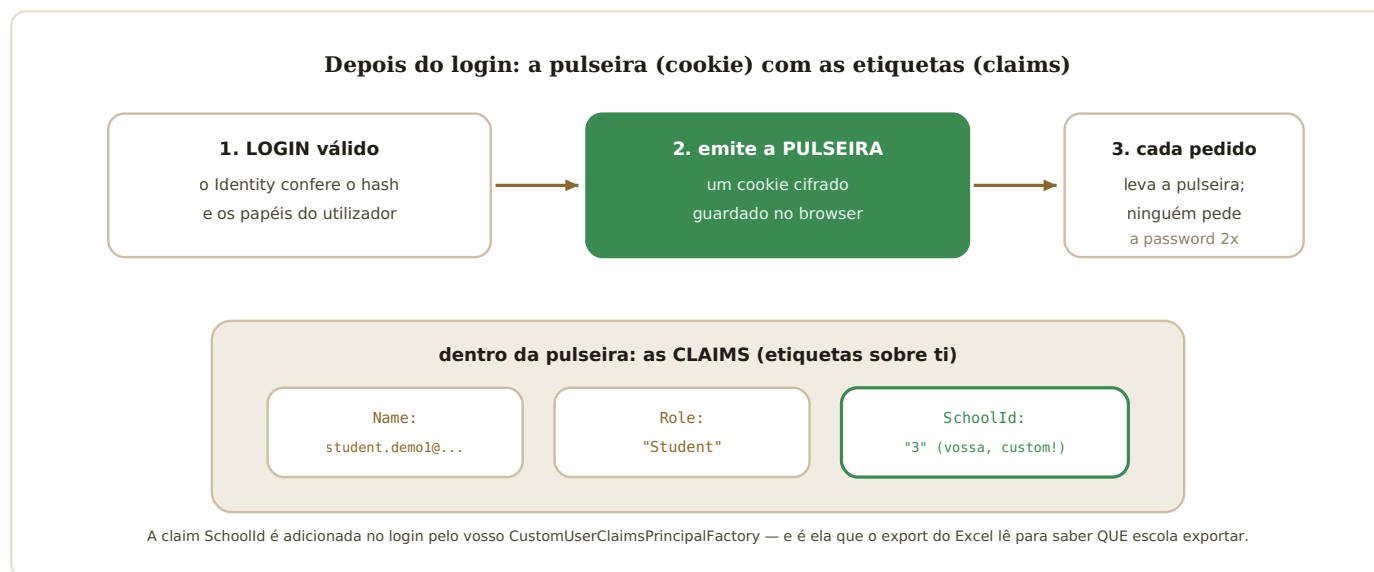


A picadora de sentido único: a base de dados nunca conhece a tua password.

O **ASP.NET Core Identity** é o sistema pronto da Microsoft para tudo o que é contas: registo, login, confirmação de email, recuperação de password, papéis e permissões. A primeira coisa a perceber é a segurança das passwords: elas *nunca são guardadas*. Guarda-se um **hash** — o resultado de uma função de sentido único: fácil de calcular, impossível de inverter. No login, o Identity calcula o hash do que escreveste e compara com o guardado.

Isto explica duas coisas do vosso projeto: porque é que o «recuperar password» envia um *link para definir nova* (não há password antiga para devolver — e é o teu `SmtEmailSender` que envia esse link!); e porque é que o `RequireConfirmedAccount = true` do `Program.cs` obriga a clicar no email de confirmação antes do primeiro login — prova que o email é teu.

19 Identity (2/2): cookie, roles e claims



O cookie é a pulseira de festival; as claims são as etiquetas que ela transporta.

Login válido não chega — a app precisa de te *reconhecer* nos pedidos seguintes. O Identity resolve com um **cookie** de autenticação: um passe cifrado que o browser guarda e envia em cada pedido, como a pulseira de um festival. Dentro vão as **claims**: afirmações sobre ti — o teu nome, o teu **role** («Student», «Teacher», «Administrator»)... e, no vosso projeto, uma claim feita à medida: o **SchoolId**, adicionada no login pelo **CustomUserClaimsPrincipalFactory** (registado no Program.cs).

Os **roles** alimentam a autorização: é o `<AuthorizeView Roles="Administrator">` do teu NavMenu a decidir que itens vês, e o mesmo nas páginas. E a claim **SchoolId** é a peça-chave do teu export: o handler lê-a do utilizador autenticado (`FindFirst("SchoolId")`) para saber que escola exportar — sem nunca confiar num valor vindo do browser, porque a claim viaja *cifrada e assinada* no cookie.

«Porque não guardar o SchoolId num campo escondido da página?»

Porque o utilizador podia alterá-lo (o browser é dele!). Na claim, vem do cookie cifrado emitido pelo servidor no login — o utilizador não o consegue forjar.

20 Injeção de dependências: tomada, ficha e quadro

Sem DI: cada página constrói tudo à mão. Com DI: pede e recebe.

SEM injeção (pesadelo)

```
var ctx = new RideReadyDbContext(...);  
var repo = new UserRepository(ctx, ...);  
var svc = new UserService(repo, ...);  
e isto repetido em CADA página...
```

COM injeção (o projeto)

```
@inject IUserService _users
```

a app constrói a cadeia toda por ti
e entrega pronta a usar

a TOMADA



IUserService (interface)

a FICHA



UserService (classe)

o QUADRO ELÉTRICO

```
AddScoped<IUserService,  
UserService>()
```

Program.cs: que ficha serve cada tomada

A DI em três peças: a interface é a tomada, a classe é a ficha, o Program.cs é o quadro.

Repara na cadeia: o `UserService` precisa de repositórios, que precisam de DbContexts, que precisam de connection strings... Se cada página construísse isto à mão, o código afogava-se. A **injeção de dependências** (DI) inverte o jogo: no arranque, o `Program.cs` regista «quando alguém pedir a interface X, entrega a classe Y» — e depois qualquer página ou serviço *pede* (`@inject` nas páginas, parâmetros do construtor nas classes) e a app entrega tudo construído, com a cadeia inteira resolvida.

A peça que torna isto possível é a **interface**: um contrato que só diz *o que* se sabe fazer (`Task<byte[]> ExportStudentsToExcelAsync(int schoolId)`), sem dizer *como*. As páginas dependem da tomada, nunca da ficha — e por isso trocar a implementação (outro export, outro sender de email, um duplo de teste) muda **uma linha no Program.cs** e zero páginas. É o coração da arquitetura do vosso projeto, e a melhor resposta à pergunta «porquê interfaces para tudo?».

21 Tempos de vida: Singleton, Scoped, Transient

Quanto tempo vive cada ficha? Os três contratos

Singleton	Scoped	Transient
UMA instância para a app inteira, sempre	uma instância POR pedido/circuito (por utilizador)	uma instância NOVA de cada vez que é pedida
<code>SmtplibEmailSender</code>	<code>serviços</code> , <code>repos</code> , <code>DbContexts</code>	(pouco usado no projeto)
ok porque não guarda estado	o normal no vosso projeto	para coisas descartáveis

Pergunta clássica: 'porque é o `SmtplibEmailSender` Singleton e os `DbContexts` Scoped?' — resposta no texto.

Singleton: um para todos. Scoped: um por utilizador/circuito. Transient: um de cada vez.

Quando a DI entrega uma «ficha», quanto tempo vive ela? São os **tempos de vida**. **Singleton**: uma única instância partilhada pela app toda — perfeito para o teu `SmtplibEmailSender`, que não guarda estado nenhum (só lê config e envia); seria desperdício criar um novo por pedido. **Scoped**: uma instância por pedido (e, no Blazor Server, por *circuito*) — é o caso de quase tudo no vosso `Program.cs`: `serviços`, `repositórios` e *sobretudo* os `DbContexts`, porque cada utilizador deve ter a sua própria conversa com a BD, isolada das dos outros. **Transient**: nova instância a cada pedido de entrega — para objetos leves e descartáveis.

A regra de ouro (e armadilha de exame): um Singleton *nunca* deve guardar dentro dele algo Scoped (como um `DbContext`) — viveria para sempre agarrado a uma conversa de BD que devia morrer no fim do pedido. O vosso projeto respeita-a: o sender Singleton só depende de `IConfiguration` e `ILogger`, ambos seguros.

22 Radzen e Serilog: as ferramentas de apoio



As duas ferramentas de apoio: componentes prontos e registo do que acontece.

Duas bibliotecas completam o quadro. O **Radzen** dá-vos componentes de interface prontos — a `RadzenDataGrid` da página de utilizadores traz paginação, ordenação e templates sem os escreverem; o `NotificationService` mostra os avisos (é ele que dá o «Falha ao exportar os alunos» no teu catch). O **Serilog** é o caderno de bordo: cada `_logger.LogInformation/LogError` espalhado pelos serviços e repositórios escreve na consola e num ficheiro diário em `Logs/` — é onde vais espreitar quando algo falha em produção, e o teu sender e o teu export escrevem lá religiosamente.

PARTE III

O RideReady por dentro — projetos, Program.cs, a cadeia real, a viagem

23 Os três projetos da solução



Receção/picadeiro, cavalariças e portão de serviço — os três projetos.

A tua solução tem três projetos com papéis distintos. O **RideReady** é a aplicação web — a receção e o picadeiro, onde os utilizadores entram, veem e clicam: todas as páginas `.razor`, o visual que construístes, e o `Program.cs` que liga tudo. A **RepositoryLibrary** são as cavalariças — a biblioteca com as entidades, os serviços, os repositórios e os dois DbContexts; é trabalho de bastidores, sem interface. E a **RideReadyAPI** é o portão de serviço: uma Web API protegida por tokens JWT, que expõe as mesmas operações a quem vem de fora — não é precisa para a app web correr, mas mostra que a lógica é reutilizável.

Este desenho responde à pergunta «porquê separar em biblioteca?»: porque a *mesma* lógica serve duas portas de entrada (web e API) sem duplicação; e porque separa quem mexe em quê — relevante numa avaliação individual.

24 A árvore de pastas (onde vive cada coisa)

```
M4_ProjectoFinal/
├─ RideReady/                               ← a app (a receção)
│   ├─ Program.cs                          ← o arranque (secção 25)
│   ├─ Services/SmtpEmailSender.cs         ← a tua feature de email
│   ├─ Components/
│   │   ├─ App.razor                       ← a raiz (head, css, js)
│   │   ├─ Layout/ MainLayout · EmptyLayout · NavMenu
│   │   ├─ Pages/ Home.razor (a tua landing)
│   │   └─ Features/
│   │       ├─ Account/ Login, Register, Manage...
│   │       ├─ Admin/Users/ AdminUsers.razor ← botão do Excel
│   │       ├─ Dashboard/ os 3 dashboards
│   │       └─ Lessons/... (área do João)
│   └─ wwwroot/ css/ js/ Images/           ← estáticos (o teu tema!)
├─ RepositoryLibrary/                      ← a biblioteca (as cavalariças)
│   ├─ Data/Context/ AppIdentityDbContext · RideReadyDbContext
│   ├─ Data/Seeds/ contas demo, dados iniciais
│   └─ Features/<área>/
│       ├─ Entities/ as classes-tabela
│       ├─ DTOs/ os objetos de transporte
│       ├─ Interfaces/ os contratos (I...Service, I...Repository)
│       ├─ Services/ a lógica ← StudentExportService aqui
│       └─ Repositories/ o acesso a dados
└─ RideReadyAPI/                          ← a Web API (o portão)
```

O padrão de arrumação chama-se **organização por features**: em vez de uma pasta gigante «Services» com tudo misturado, cada área de negócio (Users, Horses, Lessons, Purchases...) tem a sua pasta com as suas entidades, DTOs, interfaces, serviços e repositórios juntos. Vantagem prática: quando trabalhas nos utilizadores, está tudo no mesmo sítio; e numa equipa, cada pessoa «é dona» de pastas claras — outra vez a atribuição individual a funcionar a teu favor.

Decora dois endereços para a defesa: o teu `SmtpEmailSender` vive em `RideReady/Services/` (e não na biblioteca!) — é infraestrutura do anfitrião, fala com um servidor SMTP e lê a config da app; e o `StudentExportService` vive em `RepositoryLibrary/Features/Users/Services/` — é lógica de negócio reutilizável (a API também a poderia usar). Saber justificar *onde* cada coisa está vale tanto como saber o que faz.

25 Program.cs comentado: a manhã do diretor

Program.cs: a manhã do diretor da escola, em 8 gestos

1

acende as luzes e abre o caderno

Serilog: consola + ficheiro diário

2

liga o Blazor

AddRazorComponents + InteractiveServer

3

abre as duas cavalariças (BDs)

2 connection strings, 2 DbContexts

4

contrata o segurança (Identity)

cookies, roles, RequireConfirmedAccount

5

pendura a tua claim SchoolId

CustomUserClaimsPrincipalFactory

6

liga o teu email + o quadro todo

AddSingleton SmtEmailSender; AddScoped x30

7

monta as portagens (pipeline)

HTTPS → estáticos → antiforgery → páginas

8

semeia os dados demo e abre portas

SeedData.InitializeAsync + app.Run()

ordem real do vosso ficheiro — decora os 8 gestos e consegues 'ler' o Program.cs em voz alta na defesa

O arranque inteiro em oito gestos, pela ordem real do ficheiro.

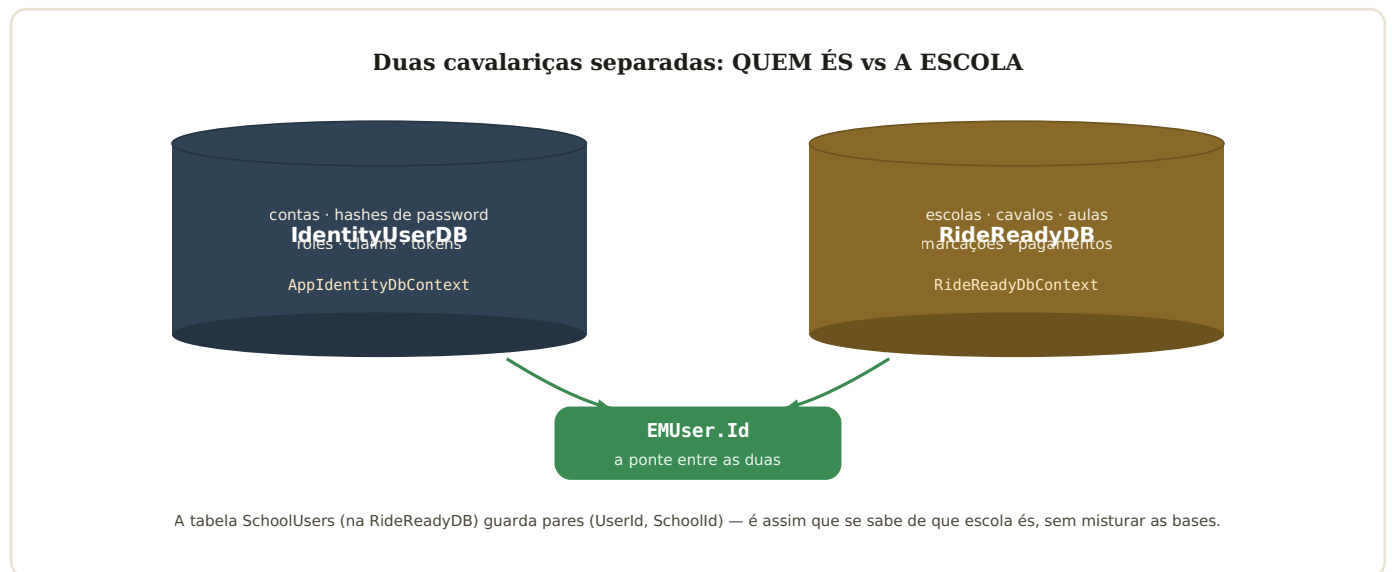
O `Program.cs` é o primeiro código a correr e lê-se como a manhã de quem abre a escola. **(1)** Acende as luzes e abre o caderno de bordo: o Serilog é configurado para escrever na consola e num ficheiro por dia. **(2)** Liga o Blazor com componentes interativos de servidor. **(3)** Abre as duas cavalariças: lê as duas *connection strings* do `appsettings.json` (com um `?? throw` que recusa arrancar se faltarem — melhor falhar alto no arranque do que misteriosamente depois) e regista os dois DbContexts, cada um com a sua tabela de histórico de migrations. **(4)** Contrata o segurança: `AddIdentityCore<EMUser>` com `RequireConfirmedAccount = true`, roles, e cookies de autenticação.

(5) Pendura a vossa etiqueta personalizada: o `CustomUserClaimsPrincipalFactory`, que mete o `SchoolId` nas claims de quem faz login. **(6)** Liga o teu `SmtEmailSender` como Singleton (a linha que substitui o sender falso do template!) e preenche o quadro elétrico inteiro — dezenas de `AddScoped<Interface, Implementação>`, incluindo o teu `IStudentExportService → StudentExportService`. **(7)** Monta as portagens da secção 10, pela ordem certa. **(8)** Em desenvolvimento, corre o `SeedData` (cria escolas, contas demo como `administrator.demo1@rideready.com`, dados de arranque) e finalmente `app.Run()` — as portas abrem e o servidor fica à escuta.

«O que acontece se a connection string faltar?»

O `?? throw new InvalidOperationException` recusa o arranque com uma mensagem clara. É intencional: uma app que arranca sem BD ia falhar de formas confusas mais tarde; assim falha já, alto e explicado.

26 As duas bases de dados e a ponte

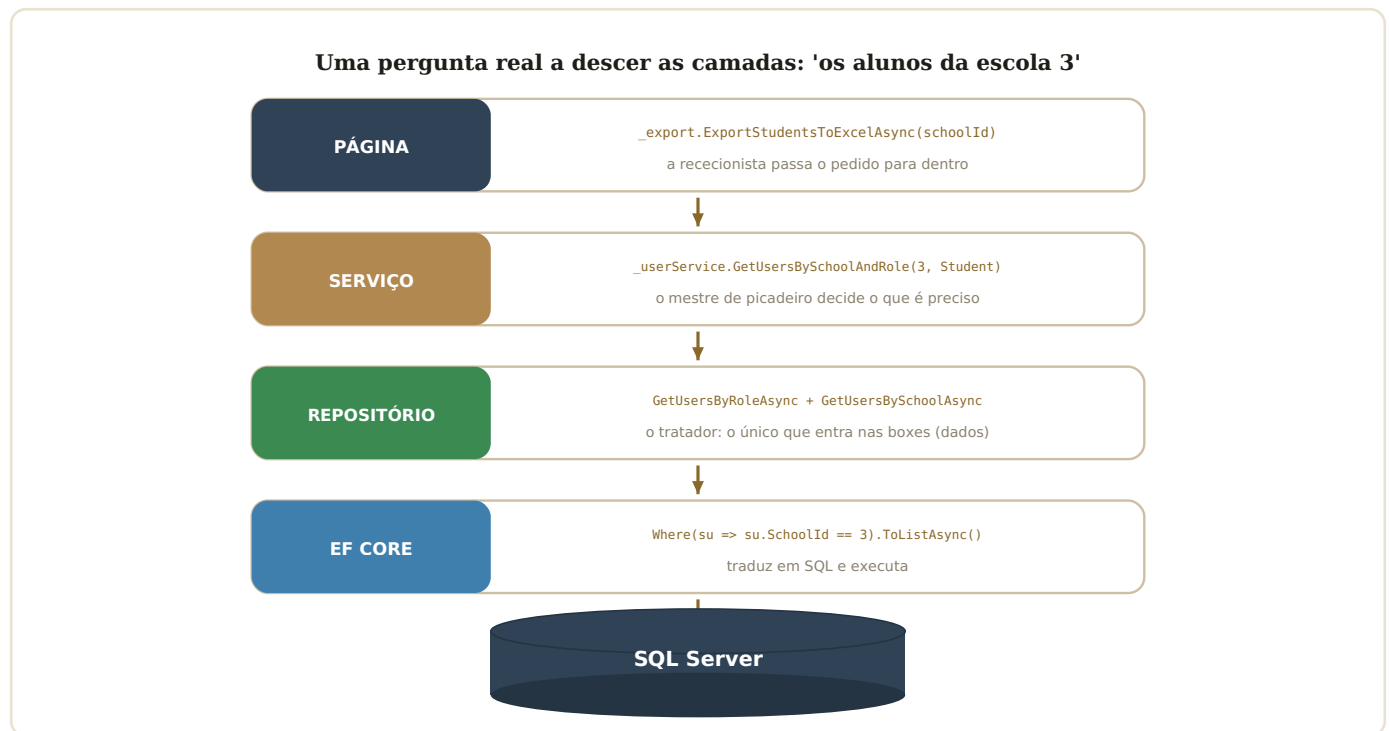


Segurança numa base, negócio na outra; o Id do utilizador faz a ponte.

A decisão de **duas bases de dados** é das mais defensáveis do projeto. A `IdentityUserDB` guarda o sensível — contas, hashes, roles — e é gerida pelas tabelas standard do Identity. A `RideReadyDB` guarda o negócio — escolas, cavalos, aulas, compras. A ponte é o `Id` do utilizador (um texto único): a tabela `SchoolUsers`, do lado do negócio, guarda pares utilizador↔escola. Vantagens a citar: isolamento do que é sensível, ciclos de vida independentes (podes fazer backup/migrar uma sem tocar na outra), e clareza de responsabilidades.

O custo, que deves admitir com à-vontade se perguntarem: perguntas que cruzam os dois mundos exigem *duas* consultas e uma junção em memória — vais ver exatamente isso, linha a linha, na cadeia da secção seguinte. É um *trade-off* consciente, não um descuido.

27 A cadeia real, linha a linha



A mesma pergunta, contada de cargo em cargo até às boxes.

Agora a cadeia inteira com o código *real*, de cima para baixo. Na **página** (`AdminUsers.razor`), o handler chama o serviço de exportação. No **serviço** (`StudentExportService`), o primeiro passo é pedir os alunos: `await _userService.GetUsersBySchoolAndRole(schoolId, StaticRole.Student)` — repara que um serviço pode usar outro serviço; é composição, não hierarquia rígida. Dentro do `UserService`, o método faz *duas* chamadas ao mundo dos dados e cruza-as:

```
public async Task<List<EMUser>> GetUsersBySchoolAndRole(int schoolId, string role)
{
    var users = await _userRepo.GetUsersByRoleAsync(role);           // 1. todos os "Student" (BD Identity)
    var schoolUsers = await _schoolUserRepo.GetUsersBySchoolAsync(schoolId); // 2. os pares user-escola (BD RideReady)

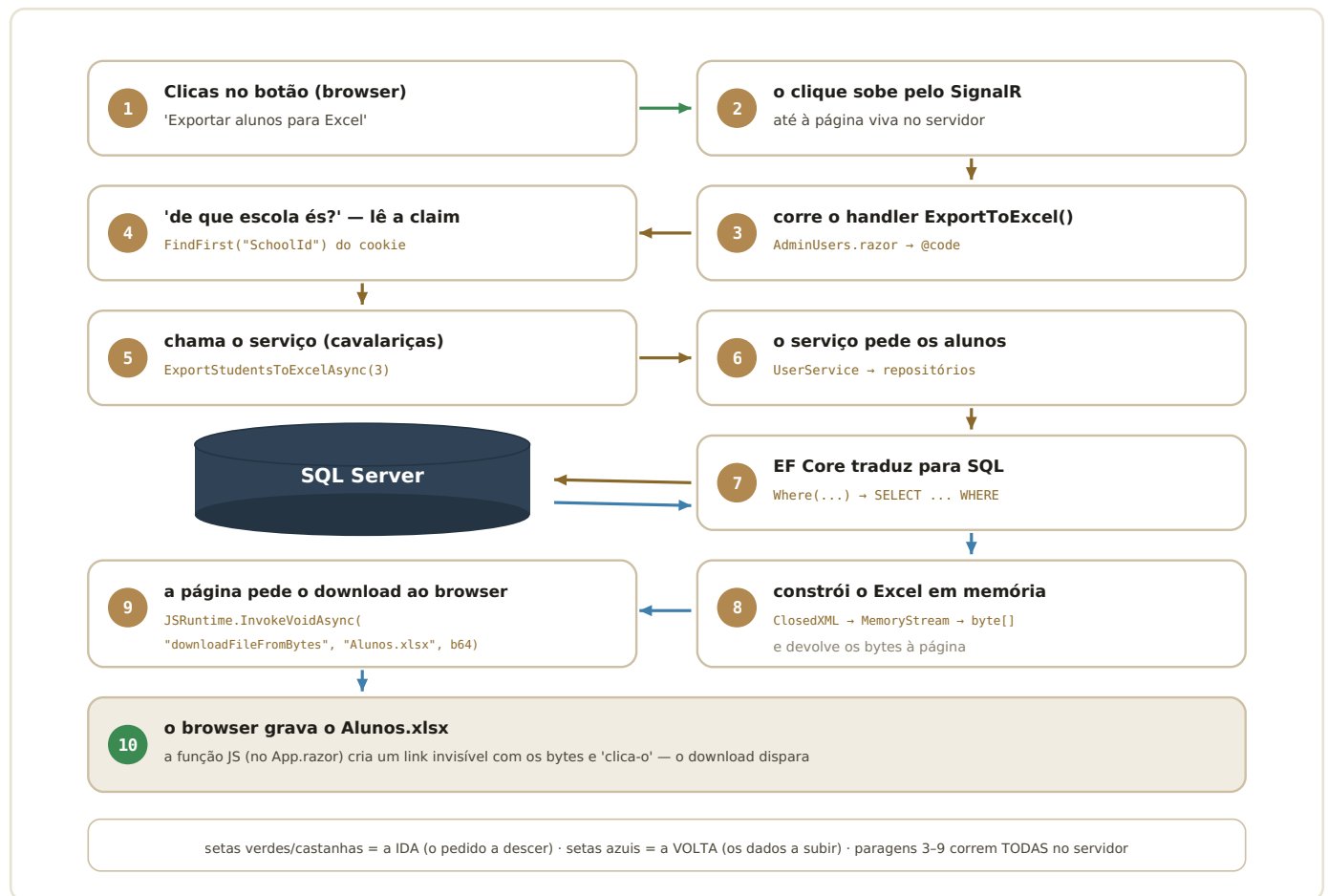
    var schoolUserIds = schoolUsers.Select(su => su.UserId).ToHashSet(); // 3. só os Ids, num conjunto rápido
    return users.Where(u => schoolUserIds.Contains(u.Id)).ToList();      // 4. interseção: Students DESTA escola
}
```

Lê as quatro linhas como uma história: **(1)** pede ao repositório de utilizadores todos os que têm o papel «Student» — isto vai à base do *Identity* (por dentro, o repositório usa o `userManager.GetUsersInRoleAsync`, a ferramenta do próprio Identity, e regista no log o que anda a fazer). **(2)** Pede ao repositório de escola↔utilizador os pares da escola 3 — isto vai à base do *negócio*. **(3)** Extrai só os Ids para um `HashSet` — um conjunto otimizado para perguntas «contém?» instantâneas. **(4)** Cruza: dos Students todos, ficam os que pertencem à escola. É a tal junção em memória que o desenho das duas BDs implica — e agora sabes apontá-la no código.

«Porquê um HashSet e não uma List no passo 3?»

Pela velocidade do Contains: numa lista, verificar se um Id lá está obriga a percorrê-la (lento se houver milhares); num HashSet é praticamente instantâneo. Com muitos utilizadores, a diferença é real.

28 A viagem do clique: o poster das 10 paragens



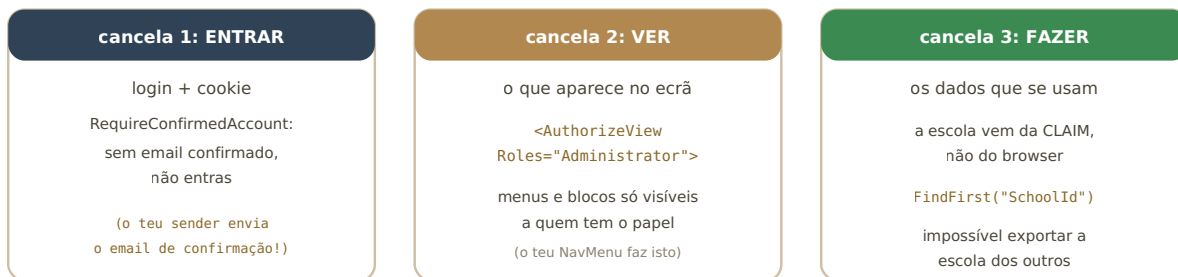
O poster: as dez paragens do clique, do rato ao ficheiro no disco.

Aqui está o fio condutor do manual inteiro, de uma ponta à outra. Repara em três coisas ao contá-lo. Primeiro, *onde* corre cada paragem: 1 no browser, 2 no fio, **3 a 9 todas no servidor** (Blazor Server!), 10 de volta no browser. Segundo, a *segurança* embutida: a escola a exportar vem da claim no cookie (paragem 4) — não de nada que o utilizador possa forjar. Terceiro, os *dois mundos de dados*: a paragem 6-7 toca nas duas bases e cruza-as, como viste linha a linha na secção 27.

Treina contá-lo em voz alta, a apontar: «clico; o clique sobe pelo SignalR; corre o meu handler; leio a claim SchoolId; chamo o serviço; o serviço pede os alunos aos repositórios; o EF traduz em SQL; o serviço monta o Excel com ClosedXML e devolve os bytes; a página pede o download ao browser por JS Interop; o ficheiro grava-se.» Dez frases — e demonstraste Blazor Server, camadas, DI, claims, EF Core e JS Interop num só fôlego.

29 Segurança na prática: as três cancelas

Três cancelas, da entrada ao picadeiro



Esconder um botão NÃO é segurança — é cortesia. A segurança real está nas cancelas 1 e 3 (e, idealmente, também em [Authorize] nas páginas).

Entrar, ver, fazer: autenticação, autorização de interface e autorização de dados.

Distingue três níveis, porque o professor distingue. **Autenticação** («quem és?»): o login, o hash, o cookie — cancela 1. **Autorização de interface** («o que vês?»): os `<AuthorizeView Roles=...>` que mostram/escondem menus e botões — cancela 2, cortesia visual. **Autorização de dados** («o que podes mesmo fazer?»): garantir no servidor que cada ação opera só sobre o que te pertence — no export, a escola sai da claim do cookie, nunca de um parâmetro vindo do browser — cancela 3, a que conta.

E uma nota de maturidade para a defesa, se for oportuno: esconder o botão não impede um utilizador malicioso de tentar chamar a ação por outra via; por isso a regra é validar sempre no servidor. É exatamente o tipo de consciência que distingue «fiz funcionar» de «percebo o que fiz».

PARTE IV

Para a defesa — perguntas, glossário, cábula

30 Vinte perguntas, vinte respostas-modelo

Vinte perguntas prováveis, com respostas-modelo curtas — para treinares em voz alta. As respostas estão escritas como tu falarias.

«Explica-me a arquitetura geral do projeto.»

São três projetos: a app Blazor Server (as páginas e o arranque), uma biblioteca com o domínio (entidades, serviços, repositórios e os dois DbContexts) e uma Web API com JWT que reutiliza a mesma biblioteca. A app segue camadas: página → serviço → repositório → EF Core → SQL Server.

«Porquê Blazor Server e não WebAssembly?»

É uma app de gestão sempre ligada e intensiva em dados; no Server, as páginas vivem no servidor ao lado dos serviços e da BD — acesso direto por injeção, sem expor lógica. O custo é a ligação SignalR permanente, aceitável neste contexto.

«O que acontece quando clico num botão?»

O clique sobe pelo circuito SignalR até à página viva no servidor; corre o handler C# do @code; este usa serviços injetados; o Blazor calcula a diferença do ecrã e devolve só isso ao browser.

«Porquê duas bases de dados?»

Separar o sensível (contas, hashes, roles — Identity) do negócio (escolas, cavalos, aulas). Cada uma tem o seu DbContext e a sua história de migrations; a ponte é o Id do utilizador na tabela SchoolUsers. O custo: cruzar os dois mundos exige duas consultas e uma junção em memória — um trade-off consciente.

«O que é a injeção de dependências e porque a usam?»

Registamos no Program.cs que classe serve cada interface; páginas e serviços pedem pela interface e a app entrega com a cadeia toda construída. Ganhamos baixo acoplamento: trocar uma implementação muda uma linha, e dá para testar com duplos.

«Porque é o SmtplibEmailSender Singleton e os DbContexts Scoped?»

O sender não guarda estado — pode ser partilhado pela app toda. Os DbContexts representam uma conversa com a BD por pedido/circuito; partilhá-los misturaria conversas de utilizadores diferentes.

«O que é uma migration? Porque há duas histórias?»

O diário de alterações ao esquema da BD; o database update aplica o que falta. Como há duas BDs, cada contexto tem a sua tabela de histórico (__EFMigrationsHistory_Identity e _RideReady), configurado no Program.cs.

«Que SQL gera a tua consulta de alunos?»

O Where com o SchoolId vira um SELECT ... FROM SchoolUsers WHERE SchoolId = @p — o filtro corre na BD. Só o ToListAsync dispara a execução; antes disso o LINQ está só a compor a pergunta.

«Como impedes um aluno de ver páginas de admin?»

Pelos roles do Identity no cookie: AuthorizeView Roles nos menus e blocos, e idealmente [Authorize] nas páginas. E nas ações, os dados sensíveis (como a escola) vêm das claims, não do browser.

«O que é a *claim* *SchoolId* e quem a põe lá?»

Uma afirmação extra dentro do cookie, adicionada no login pelo `CustomUserClaimsPrincipalFactory`. O export lê-a com `FindFirst` — assim é impossível pedir a escola de outro.

«Porque é que as páginas do Identity são SSR?»

Os serviços do Identity foram desenhados para o ciclo pedido-resposta; em `InteractiveServer` rebentam (apanhei `NullReferenceException` ao tentar). SSR para formulários de conta, `InteractiveServer` para as páginas-aplicação.

«O que faz o *Program.cs*, por ordem?»

Serilog; Blazor interativo; as duas connection strings e `DbContexts`; Identity com confirmação obrigatória; a claims factory; o registo do meu email sender e do quadro de serviços; o pipeline (HTTPS, estáticos, antiforgery, páginas); seeds em desenvolvimento; Run.

«Para que serve o *try/catch* no teu export?»

Se algo falhar (BD, geração), o catch avisa o utilizador com uma notificação de erro em vez de deixar a página rebentar em silêncio. No sender, registo o erro no log e faço throw — decisão consciente de não esconder falhas de envio.

«O que é *async/await* e porque está em todo o lado?»

Operações de I/O demoram; `await` liberta o trabalhador do servidor enquanto espera e retoma quando há resultado. Sem isso, cada espera bloquearia um thread — e corriji exatamente um caso desses ao trocar `.Result` por `await` no export.

«O que é um DTO e porque não devolver a entidade inteira?»

Um objeto de transporte só com os campos de que o ecrã precisa. Evita arrastar dados sensíveis para onde não são precisos e desacopla o ecrã do esquema da BD.

«Como funcionam as passwords?»

Nunca se guardam: guarda-se um hash de sentido único. No login compara-se o hash do que escreveste com o guardado. Recuperar password envia um link para definir nova — enviado pelo meu `SmtplibEmailSender`.

«O que é o *SignalR*?»

A ligação em tempo real entre browser e servidor que o Blazor Server usa: os eventos sobem, as diferenças de ecrã descem. Cada página aberta tem o seu circuito.

«Porque está o *SmtplibEmailSender* no projeto *RideReady* e o *StudentExportService* na biblioteca?»

O sender é infraestrutura do anfitrião (lê config da app, fala com SMTP) — não é domínio. O export é lógica de negócio reutilizável (a API também a poderia usar) — pertence à biblioteca.

«O que acontece se o servidor de email estiver em baixo?»

O `SendMailAsync` lança exceção; registo no log e faço throw — o registo do utilizador falha visivelmente. Em produção poderia optar-se por tolerar e re-tentar; é um trade-off que sei explicar.

«Se eu quisesse trocar o Excel por CSV, o que mudavas?»

Só a implementação: ou o corpo do StudentExportService, ou uma classe nova CsvExportService e uma linha no Program.cs. A página não muda — depende da interface, não da implementação. É a DI a pagar-se.

31 Glossário: 33 termos numa linha

Trinta e três termos em uma linha cada — a cábula de vocabulário. Se cada um destes te evocar o boneco certo, estás pronta.

API — porta de entrada programática: operações expostas a outros programas (a vossa usa JWT).

ASP.NET Core — a framework web da Microsoft: recebe HTTP, segurança, config, logging.

async/await — esperar sem bloquear: liberta o trabalhador durante operações lentas.

AuthorizeView — componente que mostra/esconde conteúdo conforme o papel do utilizador.

Blazor Server — interfaces em C# que correm no servidor; o browser liga por SignalR.

byte[] — ficheiro em memória: sequência de bytes em bruto (o teu Excel antes do download).

Claim — afirmação dentro do cookie sobre quem és (Role, SchoolId...).

ClosedXML — biblioteca que cria ficheiros Excel a partir de C#.

Cookie (auth) — o passe cifrado que o browser apresenta em cada pedido após o login.

DbContext — a porta para UMA base de dados; expõe tabelas como coleções C#.

DI — injeção de dependências: pedir pela interface, receber a implementação registada.

DTO — objeto de transporte: só os campos de que um ecrã precisa.

EF Core — o tradutor objetos↔tabelas (ORM); entidades, LINQ, migrations.

Entidade — classe C# que espelha uma tabela (EMUser, Horse, Lesson...).

Hash — impressão digital de sentido único; é o que se guarda das passwords.

Identity — o sistema de contas do ASP.NET Core: registo, login, roles, claims.

Interface — contrato: o QUE se faz, sem o COMO (IUserService...).

JS Interop — C# a pedir um gesto ao browser (o download do Excel).

JWT — token assinado para autenticar chamadas à API (o portão de serviço).

Lambda — mini-função em linha: su => su.SchoolId == 3.

LINQ — perguntas a coleções em C#: Where, Select, ToListAsync.

Middleware — as portagens do pipeline HTTP, pela ordem do Program.cs.

Migration — uma 'obra' registada no diário do esquema da BD.

Razor (.razor) — HTML + C# no mesmo ficheiro; @ introduz o C#.

Render mode — SSR (entrega e esquece) vs InteractiveServer (circuito vivo).

Repositório — a única camada que toca nos dados (o tratador das boxes).

Scoped/Singleton/Transient — tempos de vida na DI: por circuito / um para tudo / um de cada vez.

Serilog — o caderno de bordo: logs na consola e em ficheiro diário.

Serviço — a camada da lógica de negócio (o mestre de picadeiro).

SignalR — o fio em tempo real entre browser e servidor no Blazor Server.

SQL — a língua das bases de dados relacionais (SELECT, INSERT...).

SSR — renderização estática no servidor: gera o HTML, entrega, esquece.

Task — uma promessa de resultado futuro (devolvida por métodos async).

32 A cábula final

Se só decorares UMA página, é esta

OS 3 PROJETOS

recepção (app) · cavalariças
(biblioteca) · portão (API)
a app e a API usam a biblioteca

AS CAMADAS

página → serviço → repositório
→ DbContext/EF → SQL
cada uma só fala com a de baixo

AS 2 BASES

Identity (quem és) +
RideReady (a escola)
ponte: o Id do utilizador

BLAZOR SERVER

o C# corre no servidor;
o clique viaja pelo SignalR
SSR p/ contas; Interactive p/ app

DI

tomada (interface) + ficha
(classe) no quadro (Program.cs)
trocar implementação = 1 linha

A VIAGEM (10)

clique→SignalR→handler→claim→
serviço→repos→EF/SQL→Excel→
JS Interop→download

Os seis quadros que resumem o projeto inteiro.

Antes da defesa, revê só isto e o poster da secção 28. Se conseguires explicar cada quadro em duas frases e contar a viagem das dez paragens, tens a «parte global» segura — e os documentos do Excel e do Email tratam das tuas duas features ao pormenor. Boa defesa.